

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I Anushya devi V (192424013) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Anushya devi V

Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

ANSWER

Question 1: Smart City Surveillance System

Introduction

A Smart City Surveillance System is an advanced security system that uses multiple CCTV cameras, sensors, and intelligent software to monitor public places such as roads, railway stations, airports, shopping malls, and government buildings. The cameras continuously capture video data, which is transferred through Input/Output (I/O) devices to the computer's memory (RAM). The CPU processes the video using image processing and artificial intelligence algorithms to detect suspicious activities.

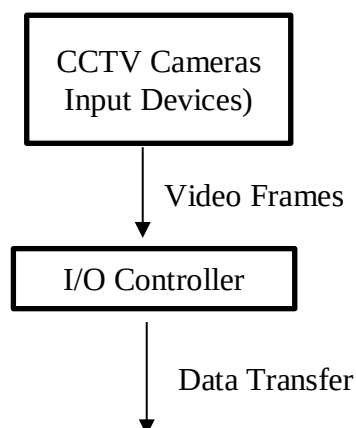
During peak hours, hundreds of cameras generate a large amount of video data every second. Since the CPU, memory, and I/O devices communicate continuously, the communication bus becomes busy, resulting in delays. Efficient bus architecture and instruction execution techniques are therefore essential for maintaining real-time performance.

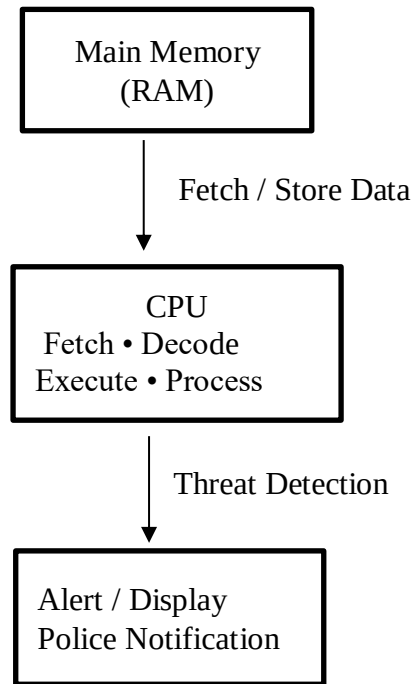
Applications

- Smart city monitoring
- Traffic management
- Crime detection
- Airport security
- Railway station surveillance
- Shopping mall security
- Military surveillance

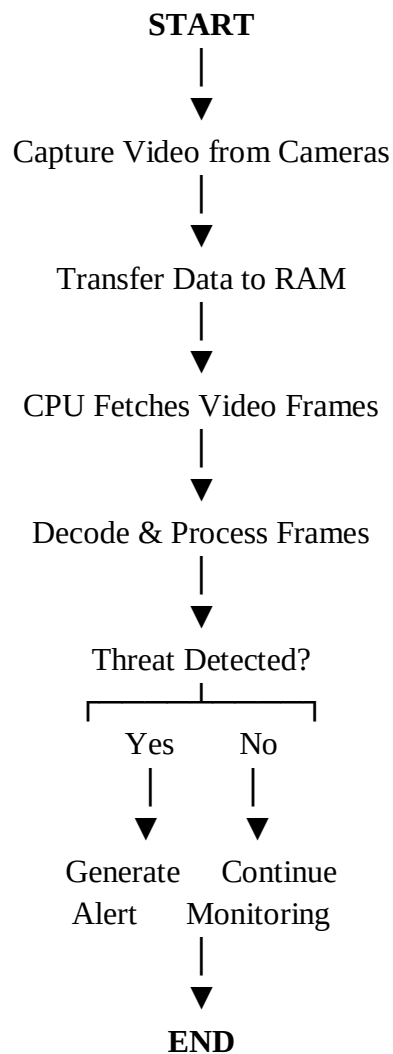
Example

Suppose 100 cameras send Full HD video simultaneously. If the CPU uses a single bus, it must wait for memory access, causing delays. A multi-bus architecture allows multiple data transfers simultaneously, reducing lag.





FLOWCHART



Related C Program (Simulation)

```
#include <stdio.h>

int main()
{
    int cameras = 5;
    for(int i=1;i<=cameras;i++)
    {
        printf("Camera %d Frame Captured\n",i);
        printf("Processing Frame...\n");
        printf("Threat Detection Completed\n\n");
    }
    return 0;
}
```

Question 2: Real-Time Stock Trading System

Introduction

A Real-Time Stock Trading System executes millions of buy and sell transactions every second. Every transaction is processed by the CPU through the Fetch–Decode–Execute cycle. When millions of instructions are executed continuously, the processor may experience a high Cycles Per Instruction (CPI) due to memory access delays and branch instructions.

High CPI increases execution time, resulting in delayed transaction processing. Modern processors reduce CPI using cache memory, pipelining, branch prediction, superscalar execution, and out-of-order execution.

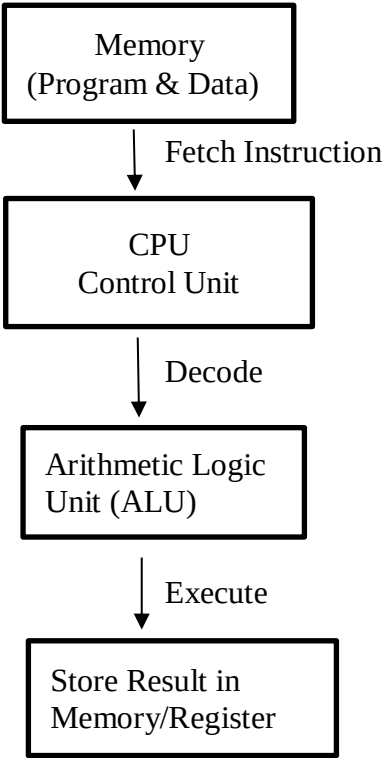
Applications

- Online stock exchanges
- Banking systems
- Online payment gateways
- Cryptocurrency trading
- Financial analytics
- High-frequency trading (HFT)

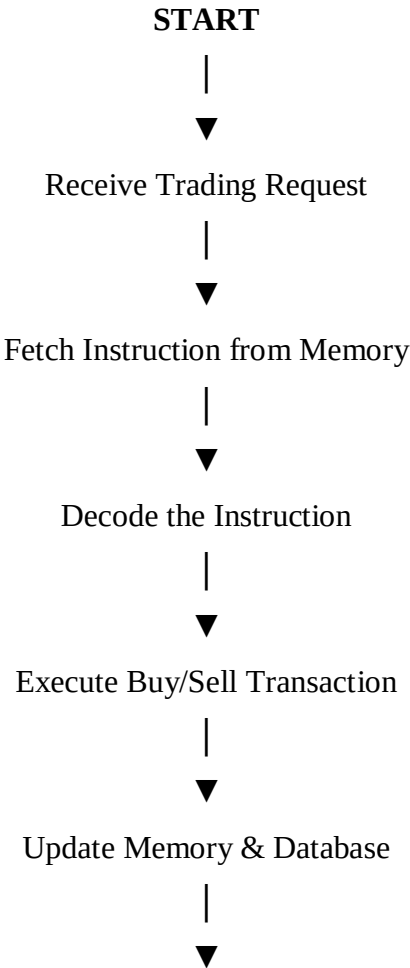
Example

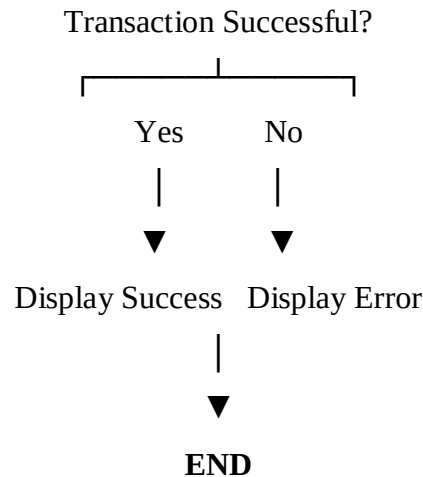
If a processor executes 10 million instructions with a CPI of 4, execution takes longer than a processor with a CPI of 2.

DIAGRAM



FLOWCHART





Related C Program

```
#include <stdio.h>
int main()
{
    int transactions = 5;

    for(int i=1;i<=transactions;i++)
    {
        printf("Transaction %d Processed Successfully\n",i);
    }
    return 0;
}
```

Question 5 – Number System Conversion in Data Communication

Introduction

In a data communication system, information is often represented in hexadecimal (Hex) format because it is shorter and easier for engineers to read than binary. However, the CPU executes all instructions in binary (0s and 1s). Therefore, hexadecimal values must be converted accurately into binary before processing.

If a software engineer incorrectly converts a hexadecimal value into binary, the CPU may execute the wrong instruction, access the wrong memory location, or transmit incorrect packet data. Efficient CPU design, error detection techniques, and benchmarking help identify and reduce these errors, improving the reliability of real-time communication systems.

Applications

- Computer networks
- Internet communication
- Routers and switches

- Embedded systems
- IoT devices
- Satellite communication
- Mobile communication
- Cybersecurity

Example

Suppose a packet contains the hexadecimal value:

Hexadecimal : 3A

Convert it into binary:

3 = 0011

A = 1010

Binary = 00111010

If it is mistakenly converted as:

00110110

the CPU may interpret it as different data or a different instruction, leading to incorrect packet processing.

C Program – Hexadecimal to Binary Conversion

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int num, i;
```

```
    printf("Enter a hexadecimal number: ");
```

```
    scanf("%x", &num);
```

```
    printf("Binary Equivalent: ");
```

```
    for(i = 31; i >= 0; i--)
```

```
    {
```

```
        printf("%d", (num >> i) & 1);
```

```
    }
```

```
    return 0;
```

```
}
```


OUTPUT

Enter a hexadecimal number: 3A

Binary Equivalent:

000000000000000000000000111010

Program Explanation

%x reads the input as a hexadecimal number.

The for loop checks each bit from the most significant bit (31) to the least significant bit (0).

The expression (num >> i) & 1 extracts one bit at a time.

The binary equivalent is displayed on the screen.

Advantages

- Easy debugging of packet data.
- Faster representation than binary.
- Reduces programming errors.
- Improves communication efficiency.
- Simplifies memory address representation.

Conclusion

Accurate conversion between hexadecimal and binary is essential for correct CPU instruction execution and reliable data communication. Modern processors use error detection, efficient memory management, and benchmarking techniques to improve performance and reduce processing errors in real-time systems.